AFRILANG Stdlib

std/c/graphdfs

Bibliothèque AFRILANG `std/c/graphdfs` (niveau complex, catégorie complex). Elle expose 7 fonction(s) : dfsOrder, dfsRec

`import "std/c/graphdfs"` · `use graphdfs`

- `dfsOrder(adj list number, n number, start number) ? list number`
- `dfsRecursiveMark(adj list number, n number) ? list number`
- `hasCycleUndirected(adj list number, n number) ? bool`
- `countComponents(adj list number, n number) ? number`
- `topoSortDfs(adj list number, n number) ? list number`
- `isTree(adj list number, n number) ? bool`
- `dfsPathExists(adj list number, n number, start number, goal number) ? bool`

Category: complex | Tier: complex | Functions: 7

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/graphdfs` (niveau complex, catégorie complex). Elle expose 7 fonction(s) : dfsOrder, dfsRec

`import "std/c/graphdfs"` · `use graphdfs`

- `dfsOrder(adj list number, n number, start number) ? list number`
- `dfsRecursiveMark(adj list number, n number) ? list number`
- `hasCycleUndirected(adj list number, n number) ? bool`
- `countComponents(adj list number, n number) ? number`
- `topoSortDfs(adj list number, n number) ? list number`
- `isTree(adj list number, n number) ? bool`
- `dfsPathExists(adj list number, n number, start number, goal number) ? bool`

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/graphdfs"
use graphdfs
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? dfsOrder(adj list number, n number, start number) ? list
? dfsRecursiveMark(adj list number, n number) ? list
? hasCycleUndirected(adj list number, n number) ? bool
? countComponents(adj list number, n number) ? number
? topoSortDfs(adj list number, n number) ? list
? isTree(adj list number, n number) ? bool
? dfsPathExists(adj list number, n number, start number, goal number) ? bool
```

4. Exemples d'utilisation

```
import "std/c/graphdfs"
use graphdfs

create result = dfsOrder(/* args */)
say result

create result = dfsRecursiveMark(/* args */)
say result

create result = hasCycleUndirected(/* args */)
say result

create result = countComponents(/* args */)
say result

create result = topoSortDfs(/* args */)
say result

create result = isTree(/* args */)
say result
```

5. Bonnes pratiques

```
? Préférez `import "std/c/graphdfs"` en tête de fichier.
? Lancez `afrilang check` avant de livrer.
? Combinez avec les modules stdlib de la même catégorie.
? Voir /stdlib/ pour le source live et les démos playground.
? Signalez les problèmes sur GitHub si une export manque.
```

6. Annexe ? source

```
module graphdfs
```

```
function dfsOrder(adj list number, n number, start number) returns list number
end

function dfsRecursiveMark(adj list number, n number) returns list number
end

function hasCycleUndirected(adj list number, n number) returns bool
end

function countComponents(adj list number, n number) returns number
end

function topoSortDfs(adj list number, n number) returns list number
end

function isTree(adj list number, n number) returns bool
end

function dfsPathExists(adj list number, n number, start number, goal number) returns bool
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/graphdfs` (tier complex, category complex). Exports 7 function(s): dfsOrder, dfsRecursiveMark, h

`import "std/c/graphdfs"` · `use graphdfs`

- `dfsOrder(adj list number, n number, start number) ? list number`
- `dfsRecursiveMark(adj list number, n number) ? list number`
- `hasCycleUndirected(adj list number, n number) ? bool`
- `countComponents(adj list number, n number) ? number`
- `topoSortDfs(adj list number, n number) ? list number`
- `isTree(adj list number, n number) ? bool`
- `dfsPathExists(adj list number, n number, start number, goal number) ? bool`

2. Installation & import

Add the module to your program:

```
import "std/c/graphdfs"
use graphdfs
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? dfsOrder(adj list number, n number, start number) ? list
? dfsRecursiveMark(adj list number, n number) ? list
? hasCycleUndirected(adj list number, n number) ? bool
? countComponents(adj list number, n number) ? number
? topoSortDfs(adj list number, n number) ? list
? isTree(adj list number, n number) ? bool
? dfsPathExists(adj list number, n number, start number, goal number) ? bool
```

4. Usage examples

```
import "std/c/graphdfs"
use graphdfs

create result = dfsOrder(/* args */)
say result

create result = dfsRecursiveMark(/* args */)
say result

create result = hasCycleUndirected(/* args */)
say result

create result = countComponents(/* args */)
say result

create result = topoSortDfs(/* args */)
say result

create result = isTree(/* args */)
say result
```

5. Best practices

```
? Prefer `import "std/c/graphdfs"` at the top of your file.
? Run `afrilang check` before shipping.
? Combine with related stdlib modules from the same category.
? See /stdlib/ for the live source and playground demos.
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module graphdfs
```

```
function dfsOrder(adj list number, n number, start number) returns list number
end

function dfsRecursiveMark(adj list number, n number) returns list number
end

function hasCycleUndirected(adj list number, n number) returns bool
end

function countComponents(adj list number, n number) returns number
end

function topoSortDfs(adj list number, n number) returns list number
end

function isTree(adj list number, n number) returns bool
end

function dfsPathExists(adj list number, n number, start number, goal number) returns bool
end
end
```